

Great. Since you're a **beginner in C#/.NET**, do **not** start coding immediately.

First understand the implementation roadmap based on your updated architecture:

- ASP.NET Core Web API
- SQL Server
- Entity Framework Core
- JWT Authentication
- Razorpay Payment Gateway
- SMTP Email
- Global Exception Middleware
- NUnit Testing
- No RabbitMQ (removed)
- 3 Databases:
 - UsersDB
 - OrdersDB
 - SupplierInventoryDB

Your uploaded LLD already contains the basic structure, but it still includes RabbitMQ references which should now be removed according to your latest decision. [\[Id 6 | Word\]](#)

STEP 1 - Install Required Software

Install

1. Visual Studio 2022

Workloads:

- ☒ ASP.NET and Web Development
 - ☒ Data Storage and Processing
-

2. SQL Server

Install:

- SQL Server Express
 - SSMS (SQL Server Management Studio)
-

3. .NET 8 SDK

Check:

```
1 dotnet --version
```

4. Postman

Testing APIs

5. Git

Version Control

STEP 2 - Create Solution Structure

Create solution:

```
1 dotnet new sln -n PharmacyManagementSystem
```

Create Projects

Auth Service

```
1 dotnet new webapi -n Pharmacy.AuthService
```

Order Service

```
1 dotnet new webapi -n Pharmacy.OrderService
```

Inventory Service

```
1 dotnet new webapi -n Pharmacy.InventoryService
```

Shared Library

```
1 dotnet new classlib -n Pharmacy.Shared
```

Add to solution

```
1 dotnet sln add Pharmacy.AuthService
```

```
2 dotnet sln add Pharmacy.OrderService
3 dotnet sln add Pharmacy.InventoryService
4 dotnet sln add Pharmacy.Shared
```

STEP 3 - Folder Structure

Inside every service:

```
1 Controllers
2 DTOs
3 Entities
4 Services
5 Repositories
6 Interfaces
7 Middleware
8 Data
9 Configurations
10 Helpers
```

Example:

```
1 AuthService
2
3 Controllers
4     AuthController
5
6 DTOs
7     LoginDto
8     RegisterDto
9
10 Entities
11     User
12     Role
13     RefreshToken
14
15 Data
16     UsersDbContext
17
18 Services
19     AuthService
20
```

21	Interfaces
22	IAuthService
23	
24	Middleware
25	GlobalExceptionHandler

STEP 4 - Install NuGet Packages

Auth Service

1	<code>dotnet add package Microsoft.EntityFrameworkCore.SqlServer</code>
2	
3	<code>dotnet add package Microsoft.EntityFrameworkCore.Tools</code>
4	
5	<code>dotnet add package</code> <code>Microsoft.AspNetCore.Authentication.JwtBearer</code>
6	
7	<code>dotnet add package BCrypt.Net-Next</code>
8	
9	<code>dotnet add package MailKit</code>

Order Service

1	<code>dotnet add package Microsoft.EntityFrameworkCore.SqlServer</code>
2	
3	<code>dotnet add package Razorpay.Net</code>

Inventory Service

1	<code>dotnet add package Microsoft.EntityFrameworkCore.SqlServer</code>
---	---

Test Project

1	<code>dotnet new nunit -n Pharmacy.Tests</code>
---	---

Add references.

STEP 5 - Database First Tables

UsersDB

Roles

```
1 CREATE TABLE Roles
2 (
3     Id INT PRIMARY KEY IDENTITY,
4     Name NVARCHAR(20) UNIQUE
5 )
```

Users

```
1 CREATE TABLE Users
2 (
3     Id INT PRIMARY KEY IDENTITY,
4     Name NVARCHAR(100),
5     Email NVARCHAR(150) UNIQUE,
6     PasswordHash NVARCHAR(255),
7     RoleId INT,
8     CreatedAt DATETIME DEFAULT GETUTCDATE(),
9     FOREIGN KEY(RoleId)
10    REFERENCES Roles(Id)
11 )
```

RefreshTokens

```
1 CREATE TABLE RefreshTokens
2 (
3     Id INT PRIMARY KEY IDENTITY,
4     UserId INT,
5     TokenHash NVARCHAR(500),
6     IssuedAt DATETIME,
7     ExpiresAt DATETIME,
8     RevokedAt DATETIME NULL
9 )
```

PasswordResetTokens

```
1 CREATE TABLE PasswordResetTokens
```

```
2  (  
3      Id INT PRIMARY KEY IDENTITY,  
4      UserId INT,  
5      TokenHash NVARCHAR(500),  
6      ExpiresAt DATETIME,  
7      UsedAt DATETIME NULL  
8  )
```

Seed Roles

```
1  INSERT INTO Roles  
2  VALUES ('ADMIN')  
3  
4  INSERT INTO Roles  
5  VALUES ('DOCTOR')
```

STEP 6 - Entity Classes

User.cs

```
1  public class User  
2  {  
3      public int Id { get; set; }  
4  
5      public string Name { get; set; }  
6  
7      public string Email { get; set; }  
8  
9      public string PasswordHash { get; set; }  
10  
11     public int RoleId { get; set; }  
12  
13     public Role Role { get; set; }  
14  
15     public DateTime CreatedAt { get; set; }  
16 }
```

Role.cs

```
1  public class Role
```

```
2 {
3     public int Id { get; set; }
4
5     public string Name { get; set; }
6
7     public ICollection<User> Users { get; set; }
8 }
```

STEP 7 - DbContext

```
1 public class UsersDbContext : DbContext
2 {
3     public UsersDbContext(
4         DbContextOptions<UsersDbContext> options)
5         : base(options)
6     {
7     }
8
9     public DbSet<User> Users { get; set; }
10
11     public DbSet<Role> Roles { get; set; }
12
13     public DbSet<RefreshToken> RefreshTokens { get; set; }
14
15     public DbSet<PasswordResetToken> PasswordResetTokens
16 { get; set; }
17 }
```

STEP 8 - JWT Configuration

appsettings.json

```
1 "Jwt": {
2     "Key": "VerySecretKey123456789",
3     "Issuer": "PharmacyAPI",
4     "Audience": "PharmacyUsers"
5 }
```

Program.cs

```

1  builder.Services
2  .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
3  .AddJwtBearer(options =>
4  {
5      options.TokenValidationParameters =
6      new TokenValidationParameters
7      {
8          ValidateIssuer = true,
9          ValidateAudience = true,
10         ValidateIssuerSigningKey = true,
11
12         ValidIssuer = builder.Configuration["Jwt:Issuer"],
13         ValidAudience = builder.Configuration["Jwt:Audience"],
14
15         IssuerSigningKey =
16         new SymmetricSecurityKey(
17             Encoding.UTF8.GetBytes(
18                 builder.Configuration["Jwt:Key"]))
19     };
20 });

```

STEP 9 - Password Hashing

Registration:

```

1  user.PasswordHash =
2  BCrypt.Net.BCrypt.HashPassword(dto.Password);

```

Login:

```

1  bool valid =
2  BCrypt.Net.BCrypt.Verify(
3  dto.Password,
4  user.PasswordHash);

```

STEP 10 - JWT Creation

```

1  var claims = new List<Claim>()
2  {
3      new Claim(ClaimTypes.NameIdentifier,

```



```
4         user.Id.ToString()),
5
6         new Claim(ClaimTypes.Role,
7             user.Role.Name),
8
9         new Claim(ClaimTypes.Email,
10             user.Email)
11     };
```

Generate Token:

```
1 var token = new JwtSecurityToken(
2     issuer,
3     audience,
4     claims,
5     expires: DateTime.Now.AddHours(1),
6     signingCredentials: credentials);
```

STEP 11 - SMTP Email

Create:

```
1 public interface IEmailService
2 {
3     Task SendEmailAsync(
4         string to,
5         string subject,
6         string body);
7 }
```

Implementation using MailKit:

```
1 var message = new MimeMessage();
2
3 message.From.Add(
4     MailboxAddress.Parse(
5     smtpSettings.Email));
6
7 message.To.Add(
8     MailboxAddress.Parse(to));
9
10 message.Subject = subject;
```

```
11
12 message.Body =
13     new TextPart(TextFormat.Html)
14 {
15     Text = body
16 };
```

Use for:

Signup

```
1 Welcome to Pharmacy System
```

Password Reset

```
1 Reset Password Link
```

STEP 12 - Global Exception Middleware

```
1 public class GlobalExceptionHandler
2 {
3     private readonly RequestDelegate _next;
4
5     public GlobalExceptionHandler(
6         RequestDelegate next)
7     {
8         _next = next;
9     }
10
11     public async Task InvokeAsync(
12         HttpContext context)
13     {
14         try
15         {
16             await _next(context);
17         }
18         catch (Exception ex)
19         {
20             context.Response.StatusCode = 500;
21
22             await context.Response.WriteAsJsonAsync(
```

```
23         new
24         {
25             Message = ex.Message
26         });
27     }
28 }
29 }
```

Register:

```
1 app.UseMiddleware<GlobalExceptionHandler>();
```

STEP 13 - Razorpay Integration

Install:

```
1 dotnet add package Razorpay.Net
```

Initiate Payment API

```
1 var client = new RazorpayClient(
2     key,
3     secret);
```

Create Order:

```
1 Dictionary<string, object> options =
2     new();
3
4 options.Add("amount", amount * 100);
5
6 options.Add("currency", "INR");
7
8 options.Add("receipt",
9     Guid.NewGuid().ToString());
1 Order order =
2     client.Order.Create(options);
```

Store:

```
1 PaymentIntent
```

record in database.

STEP 14 - Webhook Verification

Controller:

```
1 [HttpPost("webhook")]
2 [AllowAnonymous]
3 public IActionResult Webhook()
4 {
5 }
```

Verify Signature.

Update:

```
1 CREATED
2 PAID
3 FAILED
4 EXPIRED
```

exactly as in your design.

STEP 15 - Inventory Logic

At payment initiation:

```
1 Reserve stock
2 Decrease quantity_in_stock
3 Create StockReservation
```

Payment Success:

```
1 StockReservation
2     ACTIVE
3
4 to
5
6 COMMITTED
```

Payment Failure:

```
1 ACTIVE
2
3 to
4
```

```
5  RELEASED
```

Restore stock.

STEP 16 - Authorization

Admin Only

```
1  [Authorize(Roles="ADMIN")]
```

Doctor Only

```
1  [Authorize(Roles="DOCTOR")]
```

Both

```
1  [Authorize]
```

STEP 17 - NUnit Unit Testing

Example:

```
1  [Test]
2  public void Login_ValidUser_ReturnsToken()
3  {
4      Assert.IsNotNull(token);
5  }
```

Example boolean:

```
1  Assert.IsTrue(result);
```

or

```
1  Assert.IsFalse(result);
```

(as you asked previously about boolean assertions).

STEP 18 - Recommended Development Order

Do NOT jump to Razorpay immediately.

Build in this order:

Phase 1

☒ UsersDB

- ✓ JWT
 - ✓ Signup
 - ✓ Login
 - ✓ SMTP
 - ✓ Global Exception Middleware
-

Phase 2

- ✓ Drugs CRUD
 - ✓ Supplier CRUD
 - ✓ Inventory APIs
-

Phase 3

- ✓ PaymentIntent
 - ✓ Razorpay
 - ✓ Stock Reservation
 - ✓ Payment Webhook
-

Phase 4

- ✓ Orders
 - ✓ Verify Order
 - ✓ Pickup Order
 - ✓ Cancel Order
 - ✓ Scheduler for stale orders
 - ✓ Scheduler for expired payment intents
-

Phase 5

- ✓ NUnit
 - ✓ Swagger
 - ✓ Logging
 - ✓ Validation
 - ✓ Final Testing
-

My recommendation: build **Auth Service completely first** (Entities, DbContext, JWT, Signup, Login, Forgot Password, SMTP, Refresh Token). Once that is working, move to Inventory Service. Trying to implement all 3 microservices together as a beginner will become very confusing. The Auth Service is the foundation for everything else. [\[PharmacyMa...ystemHLD 2 | Word\]](#), [\[lld 6 | Word\]](#)